CENG316

SOFTWARE DESIGN DESCRIPTION

UNDERGRADUATE TRANSFER MANAGEMENT SYSTEM

G12

Group Members:

Abdullah Semih Çetin 290201093
Arda Hondoroğlu 290201015
Eren Tetik 280201013
Eray Özen 300201056
Sercan Utku Temelci 290201087
Sinan İnce 290201018
Oğuzhan Çakal 290201021
Vedat Demir 280201047

Table of Contents

1. Introduction 4
1.1. Purpose 4
1.2. Scope 4
1.2.1. Project Scope 4
1.2.2. SDD Scope 5
1.3. Context 5
2. References 5
3. Glossary 5
4. Body 5
4.1. Identified Stakeholders and Design Concerns 5
4.2. Information point 8
4.2.1. Design Concerns 8
4.2.2. Design Elements 8
4.2.3. Information Model (ER Diagram) 9
4.2.4. Design Rationale 9
4.3. Logical Viewpoint 9
4.3.1. Key Components & Responsibilities 9
4.3.2. Design Considerations & Objectives 10
4.3.3 Logical Point 12
4.4. Interaction Viewpoint 12
4.4.1. Design Rationale 12
4.4.2 Interaction Point 15
4.4.2.1 LOG.1 – Login 15
4.4.2.2 RG.1 – Register Account 16
4.4.2.3 SA.1 – Submit Application 16
4.4.2.4 UD.1 – Upload Documents 17
4.4.2.5 RV.1 – Review Application 17
4.4.2.6 VD.1 – Verify Documents 18
4.4.2.7 SFL.1 – Evaluate English Documents 19
4.4.2.8 AE.1 – Perform Academic Evaluation 20
4.4.2.9 DR.1 – Complete Department Review 20
4.4.2.10 FB.2 – Approve Final Decision 21
4.4.2.11 RP.1 – Reset Password 21
4.4.2.12 PR.1 – Publish Final Transfer Results 22
4.4.2.13 RK.1 – Approve or Adjust Ranking 22

4.4.2.14 CE.1 – Course-Equivalence (İntibak) 23
4.4.2.15 BR-INT.01 – Automatic Eligibility Check 23
5 Summary 24

1. Introduction

1.1. Purpose

The Software Design Description (SDD) document for the **Undergraduate Transfer Management System (UTMS)** serves as a comprehensive guide that outlines the structural and functional design of the system. The purpose of this document is to provide a detailed and standardized description of the software architecture, data flow, and interactions between system components to facilitate a clear understanding among developers, system analysts, and stakeholders involved in the transfer process at IZTECH.

The primary objectives of this SDD document are:

- **To define the system's architectural framework** – This document provides a structured representation of the UTMS software components, specifically focusing on the digitization of transfer workflows through logical, interaction, and information viewpoints.
- **To support system implementation** – The design specifications serve as a technical blueprint for the development team to build the automated eligibility engine and document verification modules.
- **To ensure compliance with regulations** – A key purpose is to demonstrate how the design enforces **YÖK and IZTECH undergraduate transfer regulations** and maintains **KVKK (Personal Data Protection Law)** compliance through secure data handling.
- **To ensure maintainability and scalability** – The document describes a modular design that allows the system to handle increasing numbers of applicants and adapt to future changes in transfer policies or university infrastructure.
- **To facilitate communication among stakeholders** – The SDD provides a common technical language for developers, **ÖİDB staff**, **YGK commission members**, and IT administrators at IZTECH, enabling effective collaboration.
- **To ensure system conformity to standards and best practices** – The document strictly adheres to the **IEEE STD 1016**, ensuring the software design follows international engineering best practices for software documentation.

1.2. Scope

1.2.1. Project Scope

UTMS is a centralized web platform designed to digitize and automate the entire undergraduate transfer process at IZTECH. The system supports applicants, administrative staff (ÖİDB), and academic commissions (YGK) throughout key workflow stages, including application submission, document verification, automated eligibility checks based on CGPA/quotas, and departmental evaluations.

1.2.2. SDD Scope

This SDD identifies the stakeholders and their design concerns, addressing them through multiple design viewpoints as defined by IEEE Std 1016. It details the internal software architecture, database schemas, and interaction models required to transform the requirements defined in the SRS into a functional system implementation.

1.3. Context

UTMS operates within the IZTECH IT environment and acts as a digital bridge between applicants and university administration. It interacts with:

- **Students:** Who submit digital applications and track their progress.
- **Registrar's Office (ÖİDB):** For preliminary document verification and workflow management.
- **Transfer Commissions (YGK):** For academic evaluation, course equivalency, and ranking.
- **External Data:** Adhering to YÖK regulations and IZTECH's internal database standards.

2. References

1. IEEE Std 1016-2009, IEEE Standard for Information Technology—Systems Design—Software Design Descriptions.
2. UTMS Software Requirements Specification (SRS) v2.0.
3. UTMS Problem Analysis Report v2.0.
4. UTMS Software Project Management Plan (SPMP) v1.0.

3. Glossary

- **UTMS:** Undergraduate Transfer Management System.
- **ÖİDB:** Student Affairs Registrar's Office (Öğrenci İşleri Daire Başkanlığı).
- **YGK:** Departmental Transfer Commission (Yatay Geçiş Komisyonu).
- **UI (User Interface):** The visual part of the software through which the user interacts with the system.
- **API (Application Programming Interface):** A set of protocols and tools that allow different software components to communicate and transfer data.
- **CGPA:** Cumulative Grade Point Average.
- **SDD:** Software Design Description.
- **KVKK:** Personal Data Protection Law (Kişisel Verilerin Korunması Kanunu).
- **RBAC:** Role-Based Access Control

4. Body

4.1. Identified Stakeholders and Design Concerns

Stakeholder / Role	Design Concerns & Responsibilities

IZTECH	• Approving the design document for the Undergraduate Transfer Management System (UTMS). • Authority over design alterations and compliance with YÖK and university regulations.
Transfer Students (Applicants)	• User-friendly interface for creating transfer applications, document uploads (transcripts, syllabi), and status tracking. • Handling concurrent system access during peak transfer application periods. • Authorization and secure handling of sensitive academic and personal data.
ÖİDB (Student Affairs) Staff	• Reviewing and verifying uploaded transfer documents for completeness and correctness. • Managing the initial filtering of applications before forwarding them to departments. • Overseeing final enrollment procedures for accepted students.
YGK (Transfer Commission) Members	• Evaluating academic eligibility and calculating placement scores. • Reviewing course contents for equivalence and adaptation (İntibak) processes. • Approving or rejecting applications and adding internal commission notes.
Faculty / Institute Board (FYK/EYK)	• Reviewing and finalizing the preliminary decisions submitted by the YGK. • Ensuring adherence to the department's quota limits.

Project Manager *(Abdullah Semih Çetin)*	• Overall project management and coordination of UTMS development efforts. • Defining project goals and ensuring alignment with IZTECH directives. • Ensuring timely completion of sprint tasks and feature implementations.
Database Administrator *(Sercan Utku Temelci)*	• Designing and managing the database schema for UTMS (Users, Applications, Documents, Course Mappings). • Ensuring data consistency and integrity across the transfer workflow. • Managing secure file references (S3/Cloud) for uploaded student documents.
Back-end Developers *(Arda Hondoroğlu, Eray Özen, Sinan İnce)*	• Developing and maintaining the API and server-side logic for UTMS. • Implementing complex evaluation algorithms (score calculations) and system constraints. • Ensuring secure document uploading and verification APIs.
UI/UX/Front-end Team *(Eren Tetik, Vedat Demir)*	• Designing an intuitive and accessible interface for both students and administrative staff. • Ensuring logical representation of application steps and document statuses. • Improving user engagement and error-handling in UI components. • Translating UI/UX wireframes and designs into functional, responsive React components. • Managing the global state of the application (e.g., using Redux) for complex transfer workflows. • Integrating RESTful APIs provided by the Back-end team to display dynamic application data.

Business Analyst / Requirements Eng. (Oğuzhan Çakal)	• Facilitating discussions on system design and translating AS-IS / TO-BE models into technical requirements. • Documenting business processes and UTMS workflows. • Ensuring clear communication between developers and university stakeholders.

4.2. Information point

The Information Viewpoint describes the data structures, information flow, and data management strategies within the **Undergraduate Transfer Management System (UTMS)**. It ensures that all persistent data required to support the transfer processes—ranging from student applications to departmental evaluations—is organized, secure, and compliant with institutional regulations.

4.2.1. Design Concerns

The primary design concerns addressed in this viewpoint include:

- **Data Integrity:** Ensuring that student records, transcripts, and evaluation scores remain accurate and consistent throughout the application lifecycle.
- **Regulatory Compliance:** Implementing data structures that support **YÖK/IZTECH transfer criteria** and **KVKK** requirements for handling sensitive personal information.
- **Traceability:** Maintaining a history of document versions and administrative decisions for audit purposes.
- **Relationship Management:** Defining clear associations between students, their multiple document uploads, and the multi-stage approval process (ÖİDB and YGK).

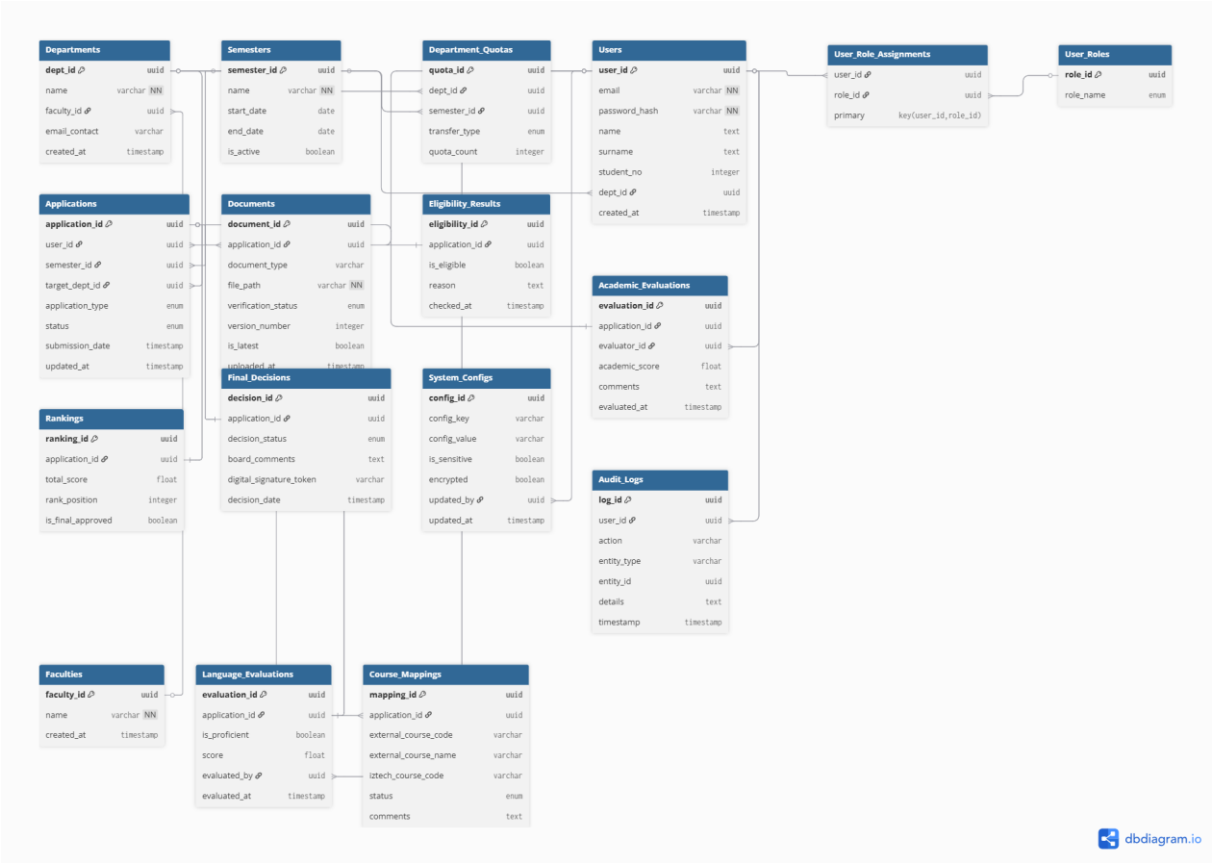
4.2.2. Design Elements

The information model of UTMS consists of several key entities as defined in the system's database scheme:

- **Users & Roles:** Manages credentials and permissions for Students, ÖİDB staff, and YGK members.
- **Applications:** The central entity that tracks the status, type, and progress of a transfer request.
- **Documents:** Handles the storage and versioning of uploaded files (Transcripts, Course Descriptions, etc.).
- **Eligibility & Evaluations:** Stores the results of automated checks and the manual academic assessments performed by commissions.
- **Audit Logs:** An immutable record of all system actions to ensure accountability.

4.2.3. Information Model (ER Diagram)

The following diagram illustrates the logical structure of the UTMS database, showing the entities, their attributes, and the relationships that govern the flow of information:



4.2.4. Design Rationale

The database design follows a **Relational Model** to ensure strict data consistency and referential integrity, which is critical for legal academic processes.

- **Normalization:** The schema is normalized to minimize redundancy, especially in handling repetitive student data and department quotas.
- **Versioning Strategy:** A separate "Documents" entity with version tracking is implemented to allow ÖİDB staff to see the history of corrected or re-uploaded files.
- **Security:** Sensitive data fields (like contact info and ID numbers) are structured to facilitate encryption and access control in line with KVKK.

4.3. Logical Viewpoint

The Logical Viewpoint illustrates the functional organization of the **Undergraduate Transfer Management System (UTMS)**, following a layered and modular architecture to ensure seamless integration with university systems. The system is organized into key functional groups to handle complex transfer workflows.

4.3.1. Key Components & Responsibilities

- **Controller Layer:**

    - Handles incoming HTTP requests from the UI, processes business logic via services, and returns structured responses.
    - Includes controllers such as **UserController**, **ApplicationController**, **DocumentController**, and **AuthenticationController**, which facilitate secure interactions between students, ÖİDB staff, and the system.
- **Service Layer:**
    - Encapsulates the core business logic and provides methods for managing transfer applications, automated eligibility checks, and document verification.
    - Includes **UserService**, **ApplicationService**, **EligibilityService**, and **AuthService**, ensuring smooth communication between the web controllers and the data persistence layer.
- **Model Layer (Data Layer):**
    - Defines core data structures used to store and process student records, transfer applications, evaluation scores, and uploaded files.
    - **Key models include:**
        - **User (UserID, Name, Email, Role, Password):** Represents all system users (Students, ÖİDB, YGK members).
        - **Application (AppID, Type, Status, CreatedAt, UpdatedAt):** Manages the lifecycle of transfer requests (e.g., Central Placement vs. Lateral Transfer).
        - **Document (DocID, FileName, FileType, UploadedBy):** Handles metadata for transcripts, course descriptions, and identity files.
        - **Token (TokenID, UserID, Expiration):** Ensures stateless authentication and session security.
- **File Server & Storage:**
    - Manages the physical storage of sensitive documents, including digital transcripts and departmental equivalence lists.
    - Utilizes **FileService** for categorized uploading, secure retrieval, and protection of student files.
- **Database Layer:**
    - A structured collection of transfer records, evaluation criteria, and user information.
    - Ensures efficient data retrieval for ranking students and enforces security policies for different user roles.
- **External Integrations:**
    - **IZTECH Auth / UBYS API:** Facilitates student record validation and ensures that transfer applicants meet basic institutional criteria.
- **JWT (JSON Web Token):** Manages secure session validation and role-based access for API endpoints.

4.3.2. Design Considerations & Objectives

- **Data Integrity & Consistency:** The system enforces Role-Based Access Control (RBAC) to ensure that transfer records and commission decisions are accurate and tamper-proof.

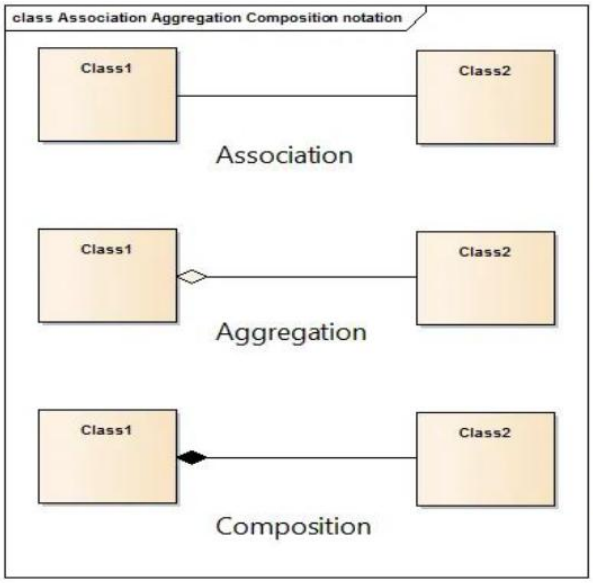
- **Efficiency & Performance:** The layered structure optimizes query performance for tracking high volumes of transfer applications and evaluating eligibility against department quotas.

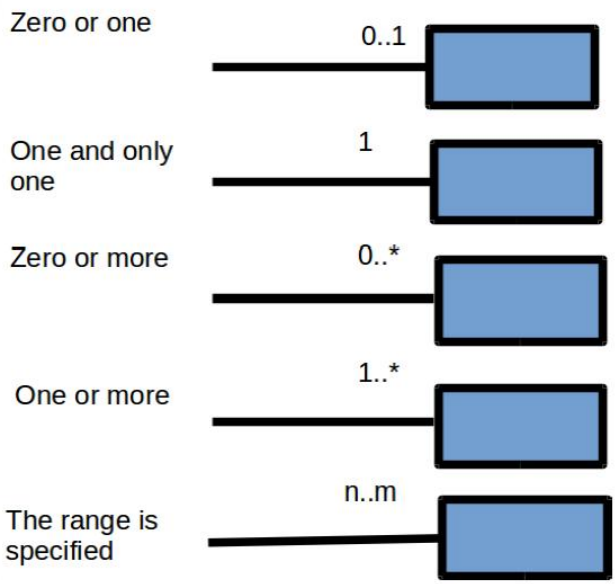
- **Security & Authentication:** Integration of JWT-based authentication ensures that personal student data is protected during the verification and evaluation stages.

- **Scalability & Maintainability:** The modular design allows for future extensions, such as adding new transfer types or modifying evaluation rules without affecting the core architecture.

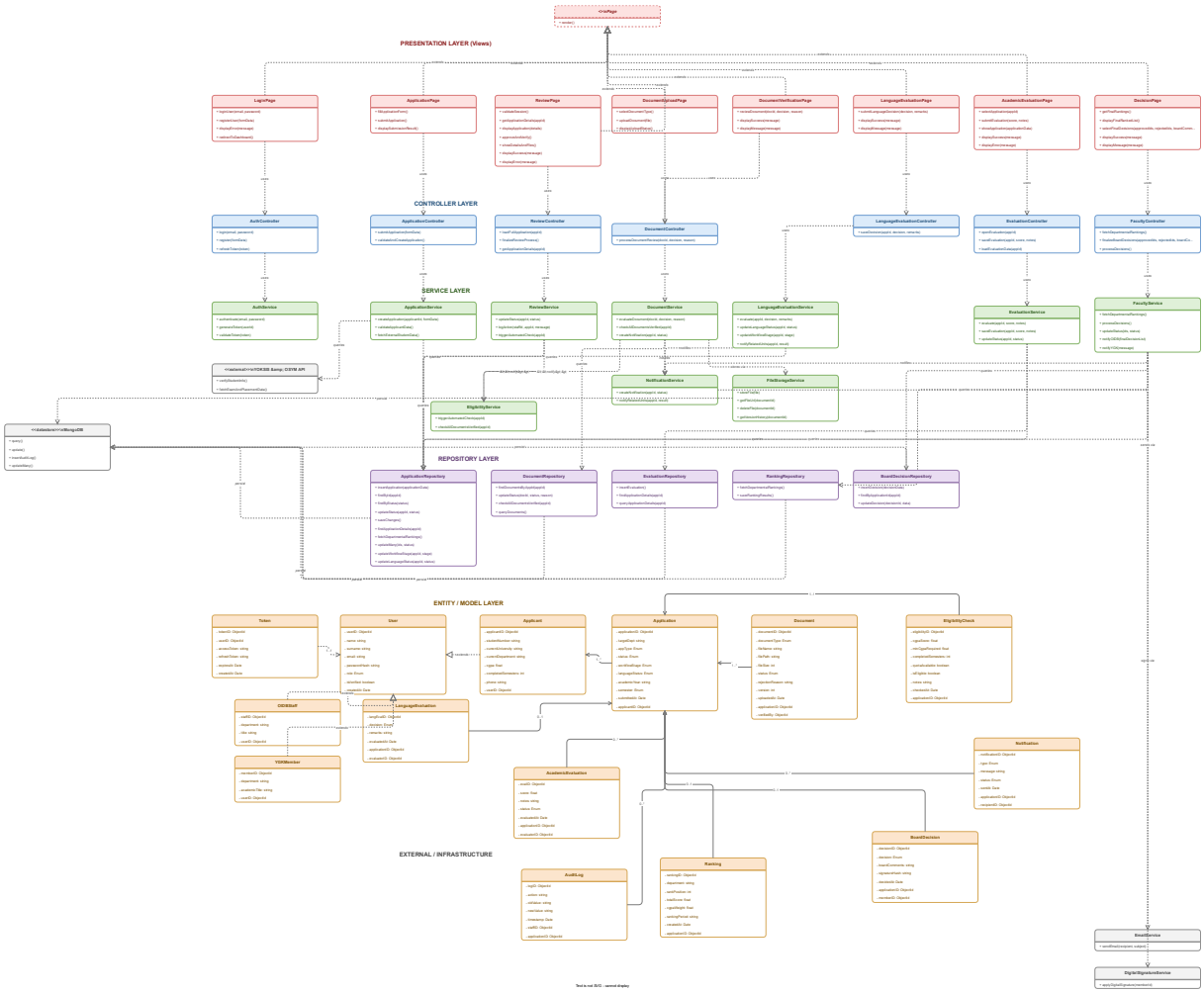
- This structured **logical architecture** provides a **reliable, secure, and efficient undergraduate transfer management system**, ensuring a seamless experience for students, advisors, and university staff.
  Used Notations:
  Member Access Modifiers
  All classes have different access levels based on the **visibility modifier:**
  1- Public (+)
  2- Private (-)
  3- Protected (#)
  4- Package (~)

Notation:





4.3.3 Logical Point



4.4. Interaction Viewpoint

4.4.1. Design Rationale

The architectural and technological decisions for the **Undergraduate Transfer Management System (UTMS)** are made to ensure scalability, maintainability, and seamless integration with university systems. This section outlines the rationale behind each major technology and design pattern used in the system:

1) React + Vite (Frontend Framework)

React was chosen due to its component-based architecture, which allows for reusable, maintainable, and modular user interfaces. Combined with Vite, it provides a significantly faster development environment and optimized build performance. Compared to alternatives like plain JavaScript or jQuery, React:

- Provides a virtual DOM for faster rendering and better performance.
- Enables the use of reusable components, simplifying large-scale UI development for complex transfer workflows.

- It has a large ecosystem and community support. Alternatives like Angular or Vue were considered, but React's flexibility and the enhanced developer experience provided by Vite made it a more suitable choice for this project.

2) Zustand (State Management)

Zustand is used for global state management across the application, especially for managing user sessions, application statuses, and shared data during the multi-step transfer process.

- Zustand provides a lightweight and performant state container, reducing boilerplate compared to Redux.
- Centralized state management simplifies communication between unrelated components, such as document upload status and eligibility displays. While React's Context API could be an alternative for simpler state needs, Zustand offers better scalability and structure for managing the complex states involved in UTMS.

3) node-fetch (HTTP Client)

node-fetch is used for handling server-side API calls due to its lightweight nature and adherence to the Fetch API standard.

- Provides a consistent, promise-based API for both internal and external requests.
- It makes it easier to manage request headers and handle responses from the UBYS API.

4) Node.js (Backend Runtime)

Node.js is used as the backend environment due to its non-blocking, event-driven architecture.

- Allows high performance and efficiency when handling concurrent system access during peak application periods.
- Enables a unified JavaScript-based stack, simplifying the development process between frontend and backend. Compared to other runtimes, Node.js provides a faster development environment particularly suited for real-time validation and high-volume data processing.

5) Drizzle ORM (TypeScript ORM)

Drizzle ORM is selected as the primary tool for interacting with the **PostgreSQL** database due to its "TypeScript-first" philosophy and performance benefits.

- **Type Safety & SQL-like Syntax**: Unlike heavy ORMs, Drizzle provides a thin abstraction that stays close to SQL while ensuring full compile-time type safety for complex student and application records.

- **No Hidden Overheads**: It eliminates the performance bottlenecks of traditional ORMs by avoiding heavy class-based abstractions, which is critical for handling high concurrent access during peak transfer periods.

- **Automated Migrations**: Drizzle Kit allows the development team to generate and manage SQL migrations automatically, ensuring the schema stays in sync with the **IZTECH/YÖK regulation** changes.

- **Relational Mapping**: It provides a clear way to define the one-to-many and many-to-many relationships found in the UTMS ER diagram (e.g., Users to Applications, Applications to Documents) without the overhead of a traditional Data Mapper.

6) Relational Database (PostgreSQL)

A **Relational Database (PostgreSQL)** is implemented instead of a NoSQL solution to ensure strict data integrity and regulatory compliance:

- **ACID Compliance**: Ensures that critical transfer operations—such as ranking calculations and final board approvals—are atomic and consistent.

- **Relational Integrity**: The system uses **Foreign Keys** and **Check Constraints** to enforce business rules, such as ensuring a document always belongs to a valid application.

- **Structured Querying**: Relational databases are optimized for the complex JOIN operations required to aggregate student data, departmental quotas, and evaluation scores for the ranking engine.

- **KVKK Compliance**: PostgreSQL provides robust mechanisms for field-level encryption and role-based access control, ensuring sensitive personal data is handled securely.

.

7) JWT (Authentication)

JWT (JSON Web Token) is used for user authentication and session handling.

- Stateless authentication, reducing server-side session storage requirements.
- Easily integrated into HTTP headers for secure API calls across the system.
- Enables role-based access control (RBAC) using payload claims for students, ÖİDB staff, and YGK commission members. Traditional session-based authentication was avoided due to the system's API-first design and the need for high scalability.

8) UBYS API (Integration Layer)

The system integrates with the University's backend systems (UBYS) via APIs for:

- Fetching existing student records to validate eligibility.
- Synchronizing academic data to ensure applicants meet institutional criteria. Instead of duplicating data, real-time API integration ensures data consistency across platforms and reduces maintenance overhead.

9) MVC Architecture (Design Pattern)

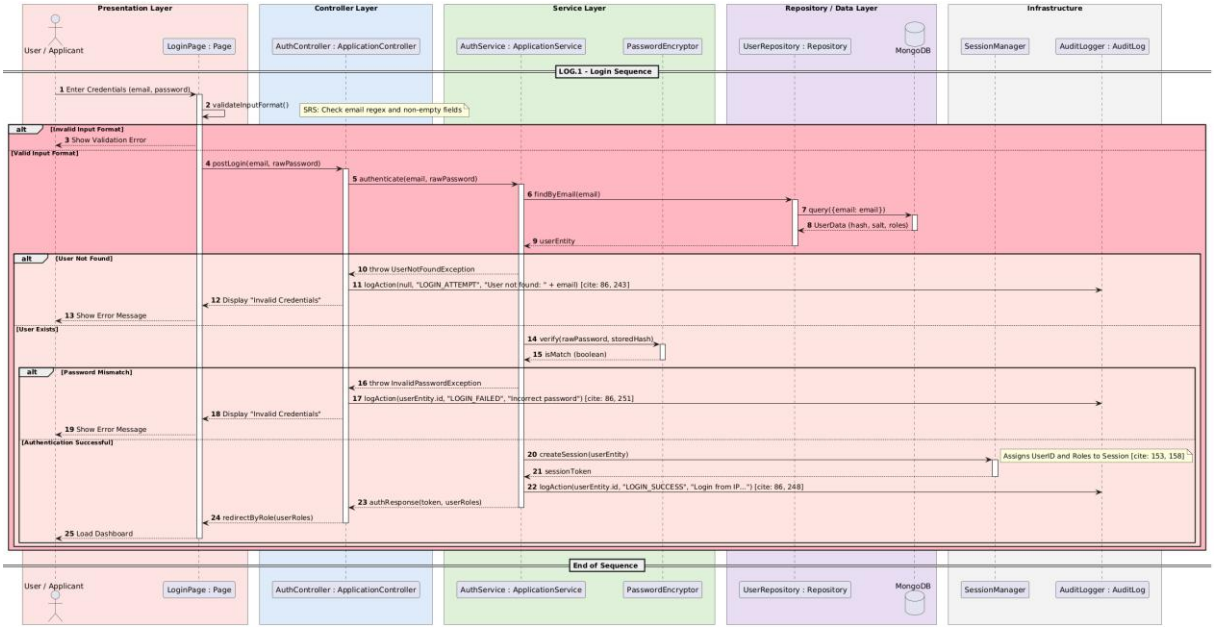
The system is structured using the Model-View-Controller (MVC) pattern.

- **Model layer** handles data and business logic (via Mongoose).
- **Controller layer** handles HTTP requests and processes business logic via services.

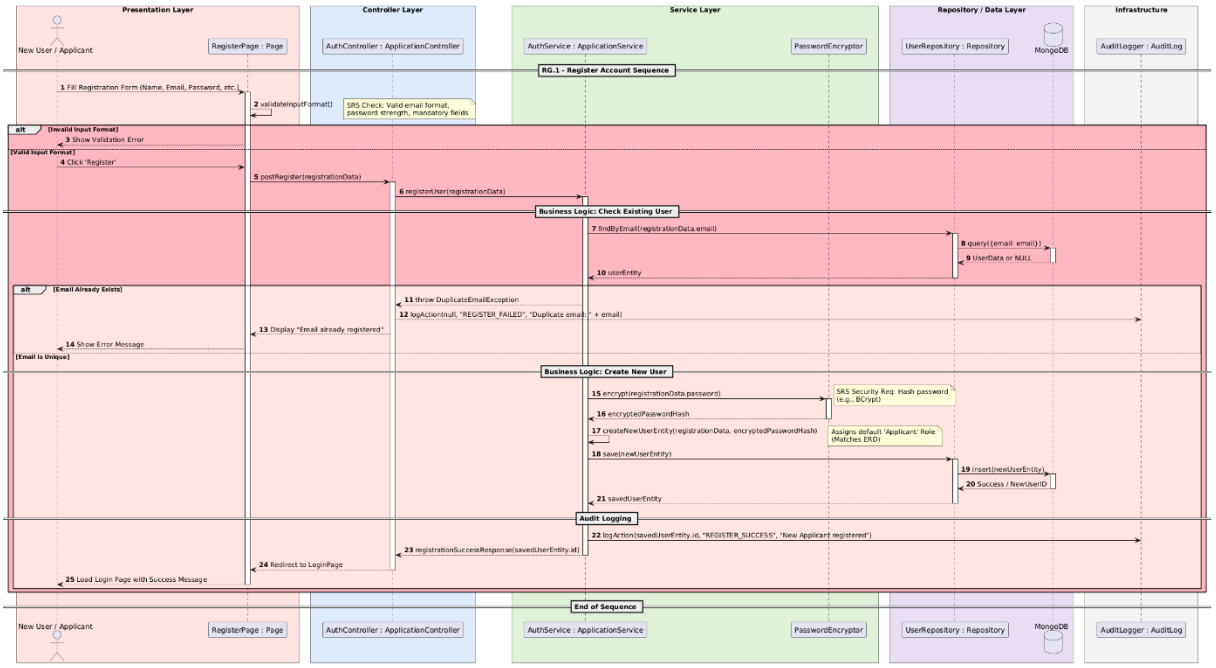
View layer is managed separately by the frontend (React). MVC improves modularity, makes the codebase more maintainable, and separates responsibilities clearly between evaluation logic and user interactions

4.4.2 Interaction Point

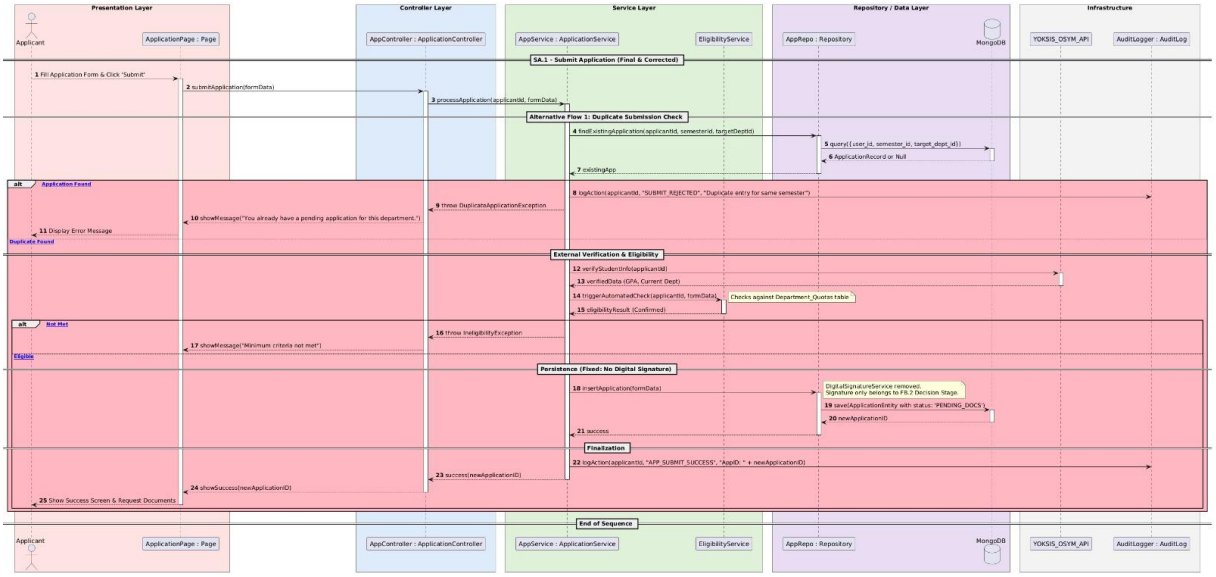
4.4.2.1 LOG.1 – Login



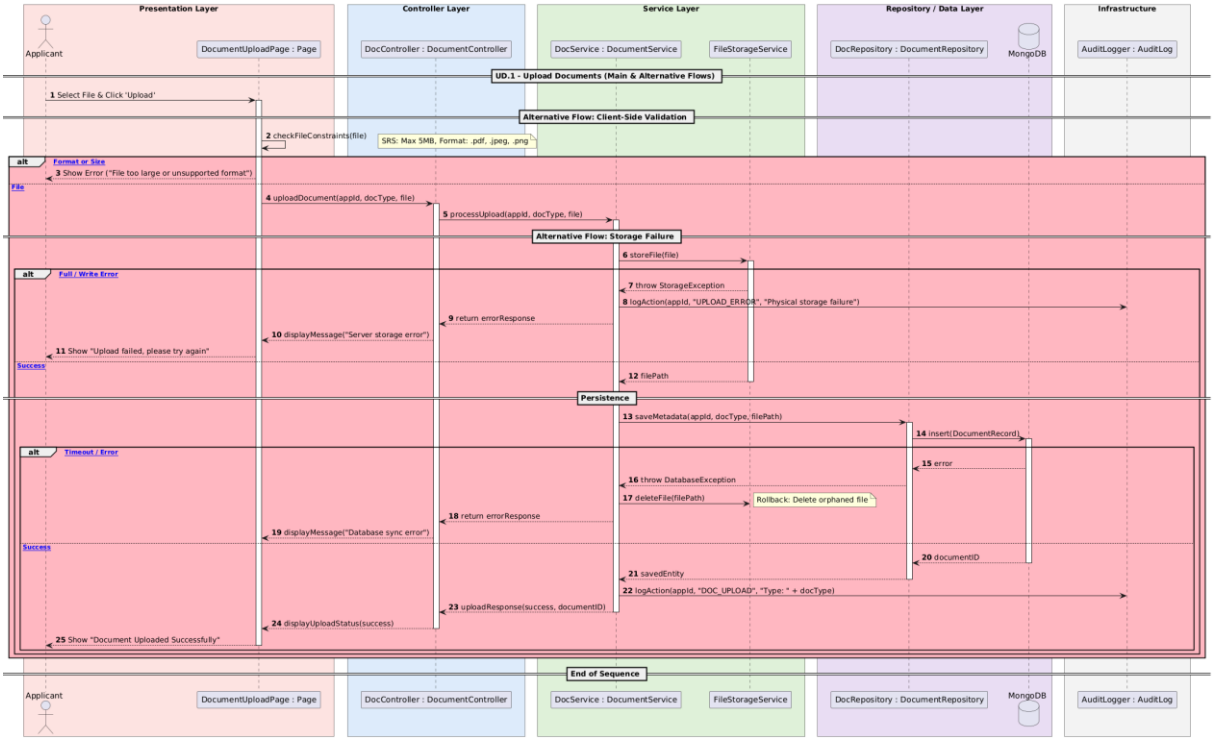
4.4.2.2 RG.1 – Register Account



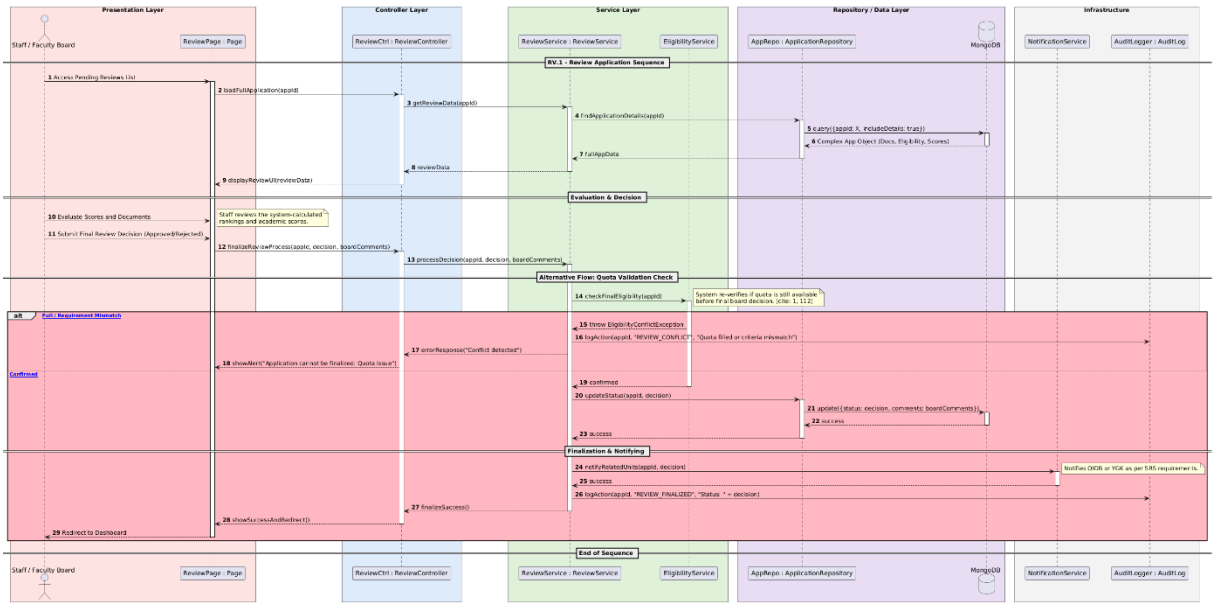
4.4.2.3 SA.1 – Submit Application



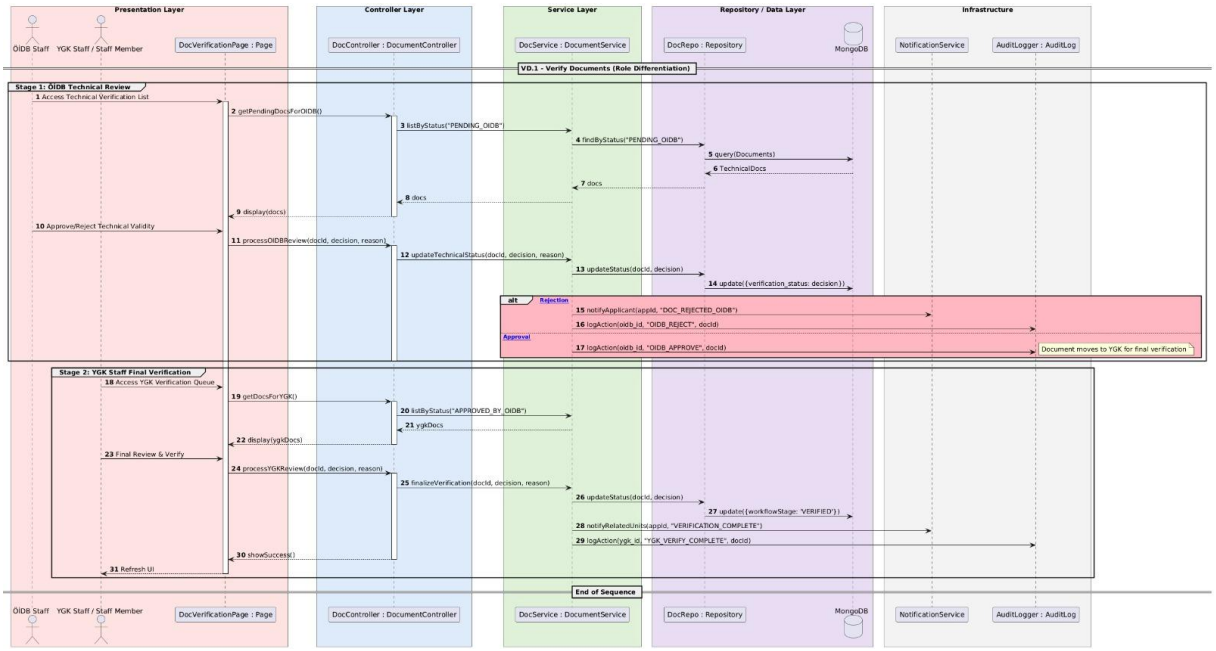
4.4.2.4 UD.1 – Upload Documents



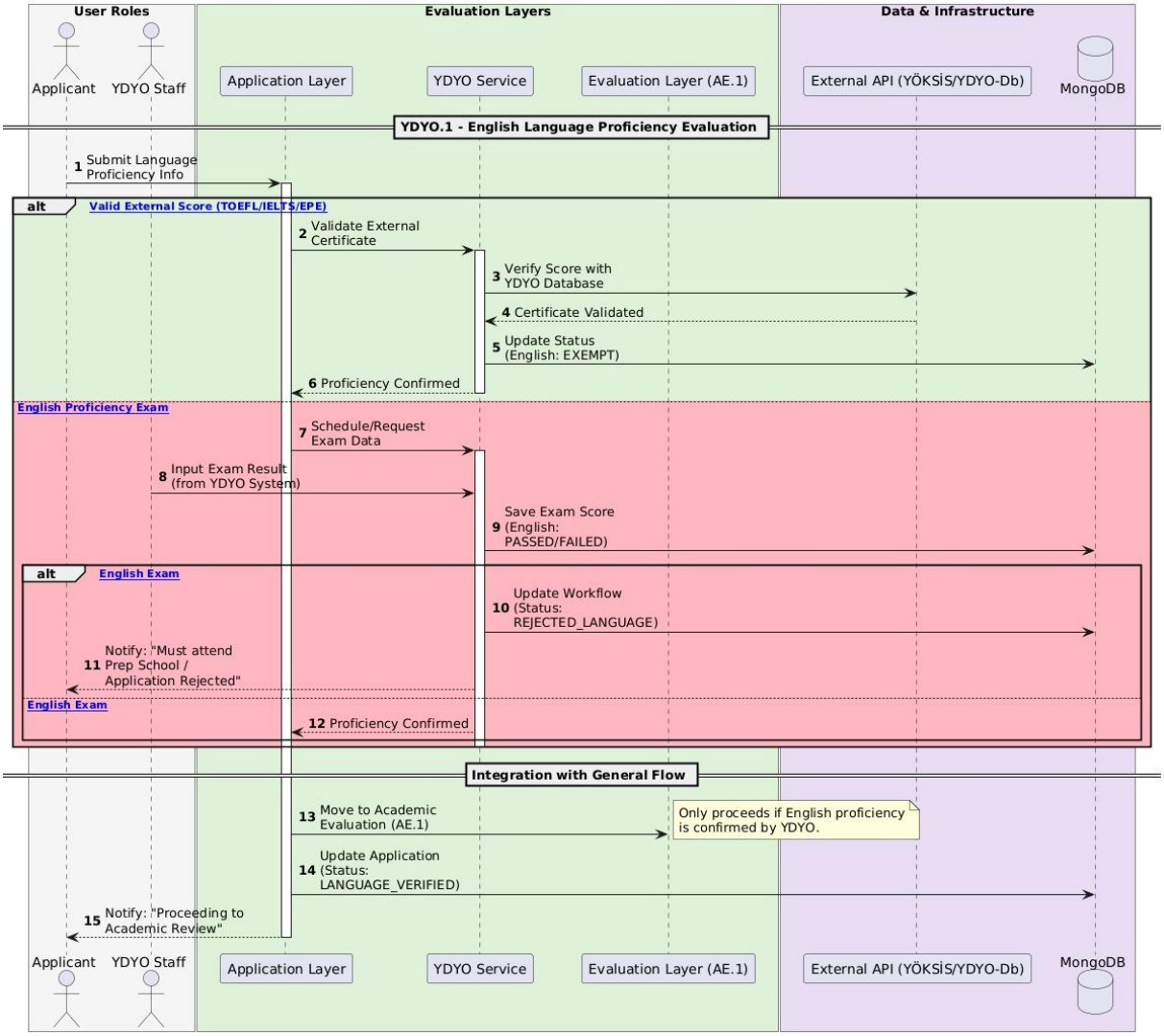
4.4.2.5 RV.1 – Review Application



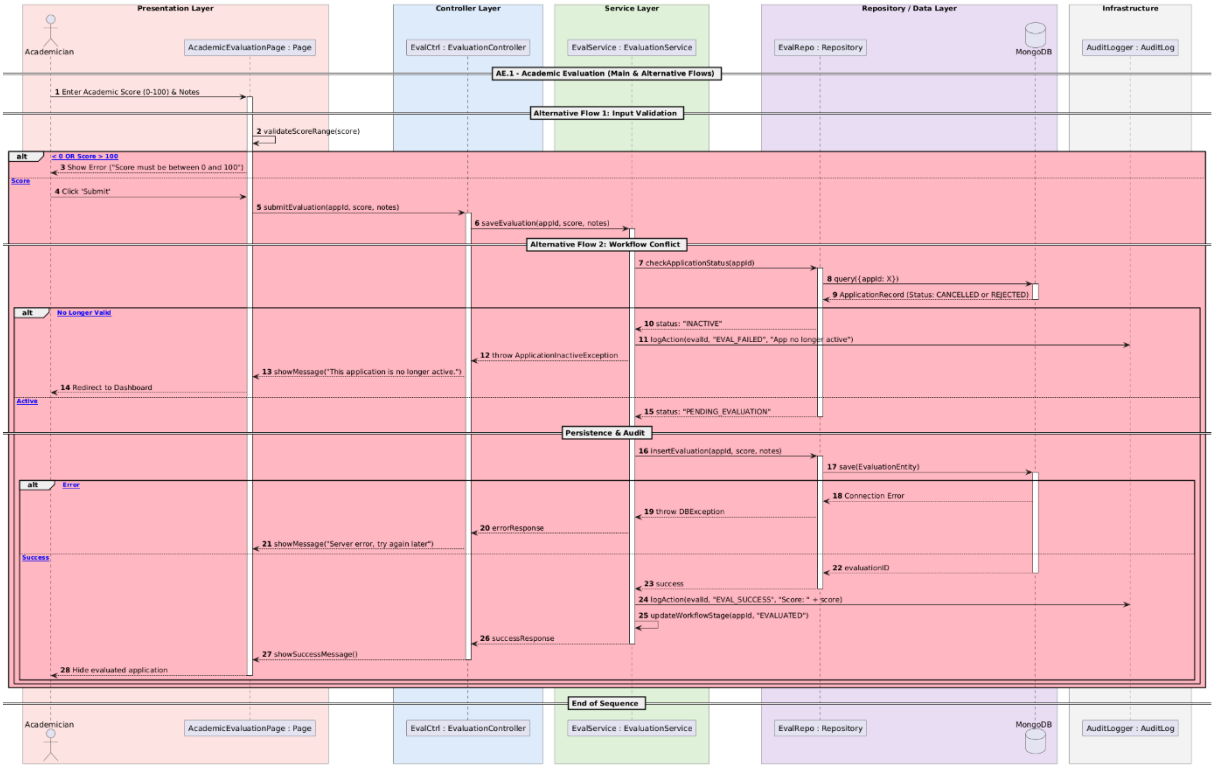
4.4.2.6 VD.1 – Verify Documents



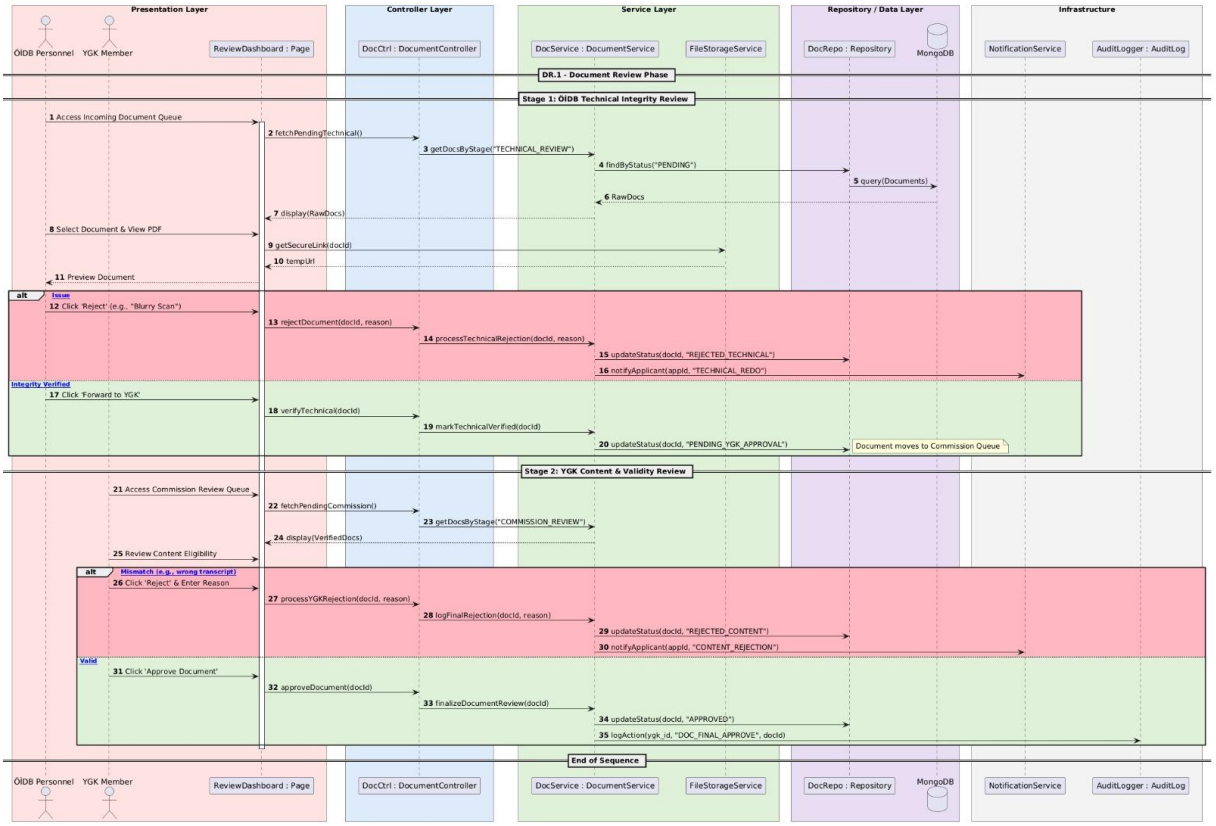
4.4.2.7 SFL.1 – Evaluate English Documents



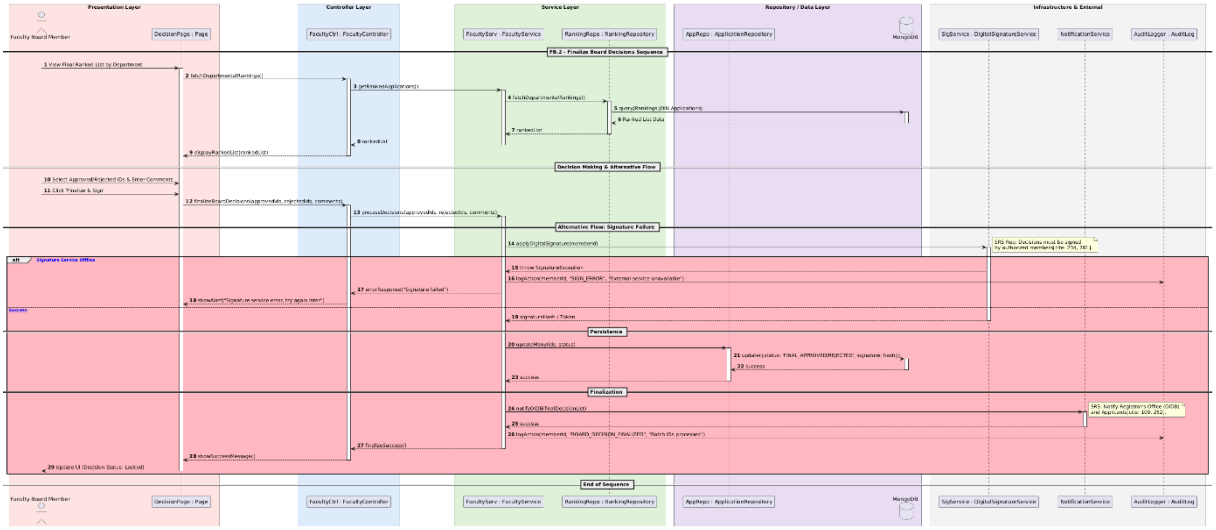
4.4.2.8 AE.1 – Perform Academic Evaluation



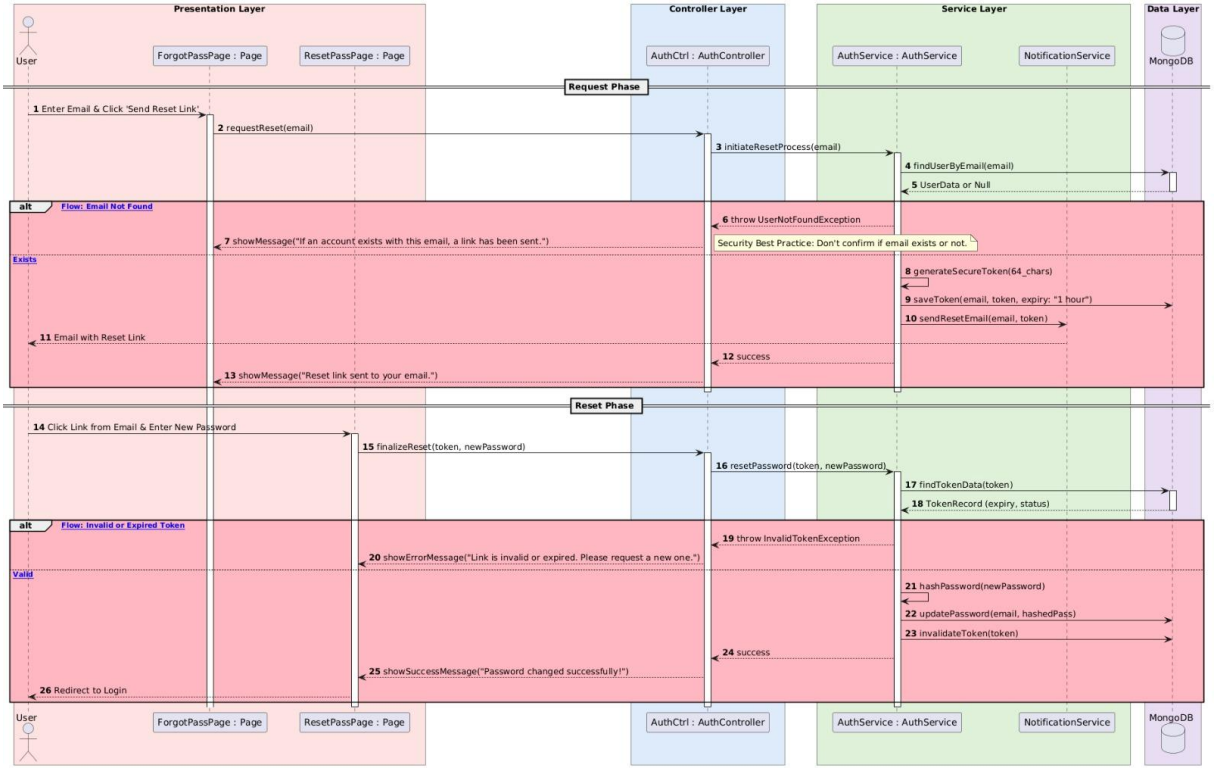
4.4.2.9 DR.1 – Complete Department Review



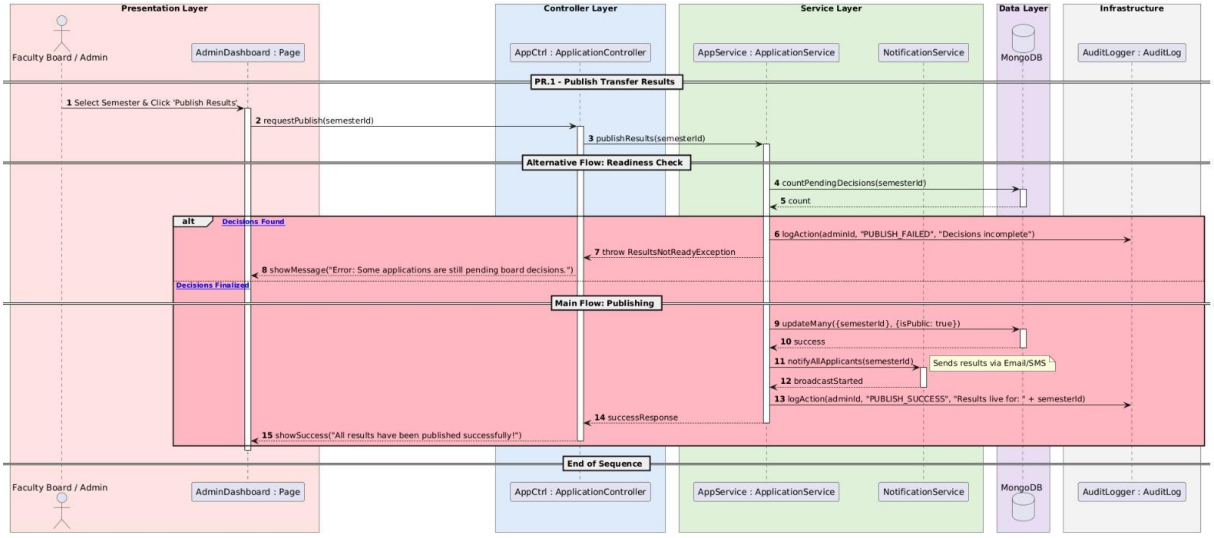
4.4.2.10 FB.2 – Approve Final Decision



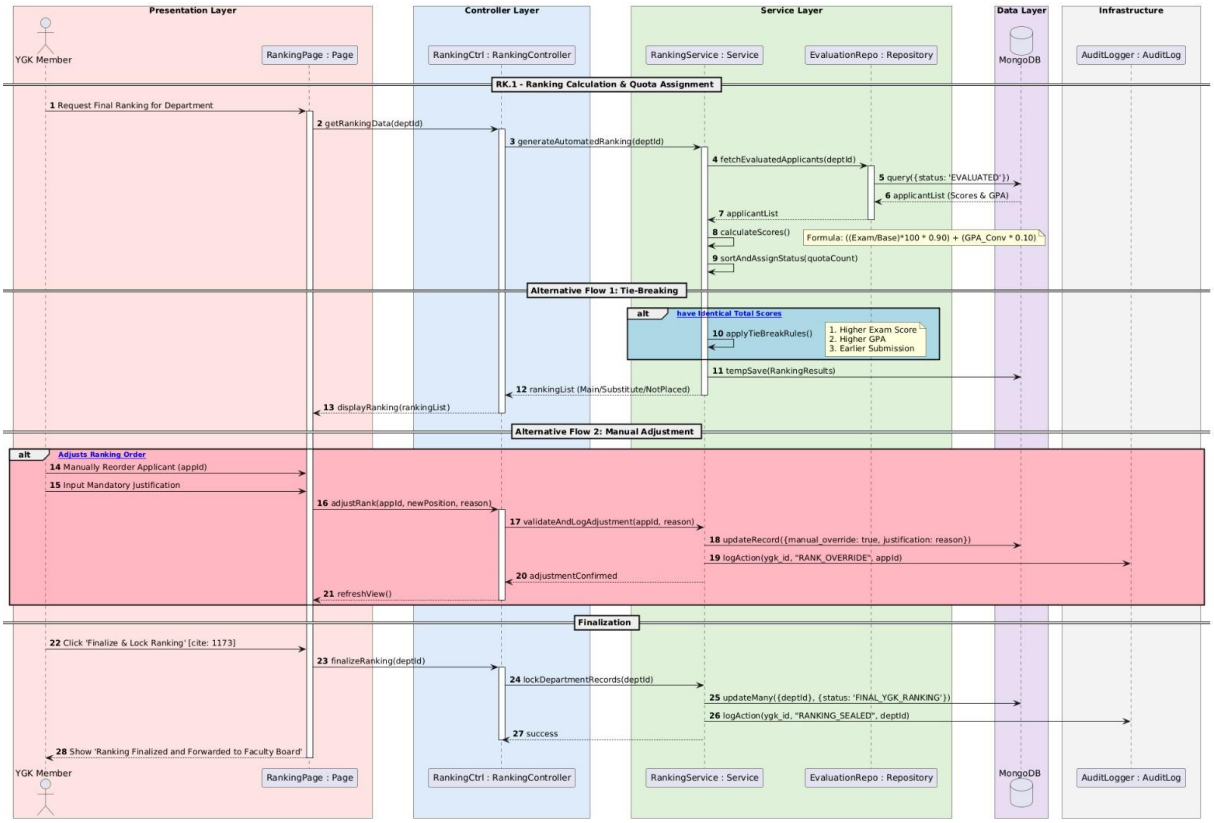
4.4.2.11 RP.1 – Reset Password



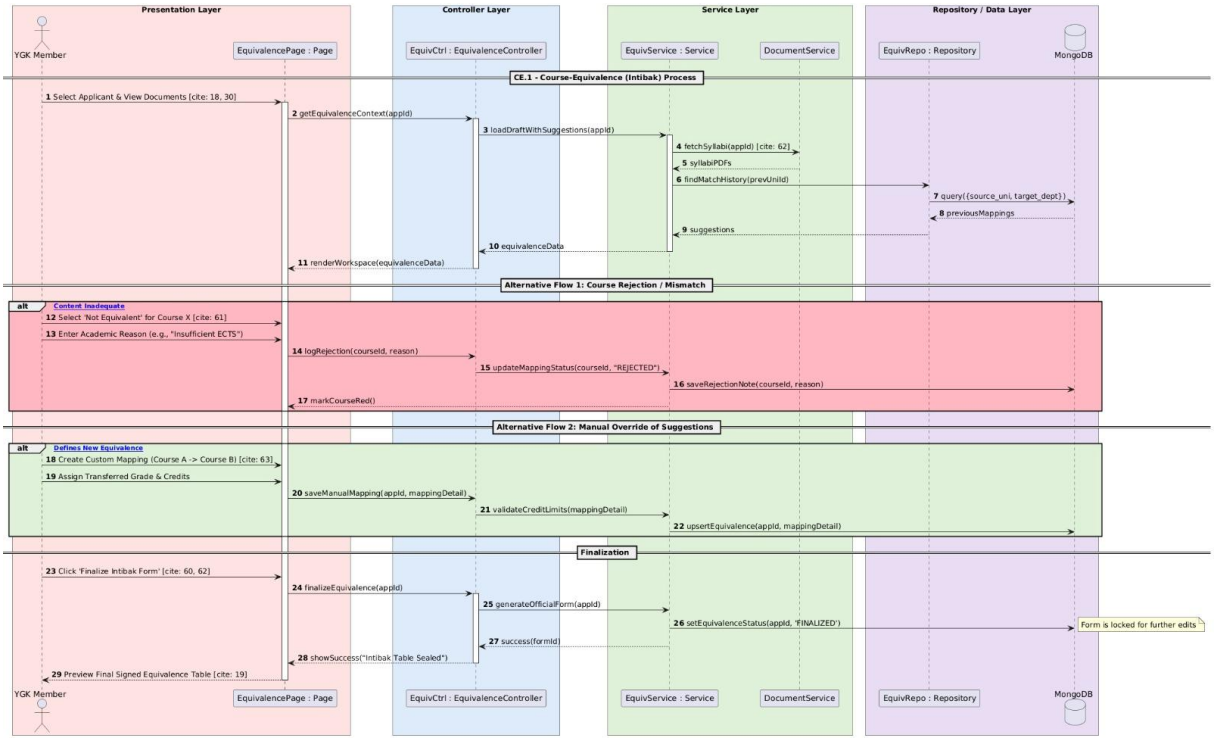
4.4.2.12 PR.1 – Publish Final Transfer Results



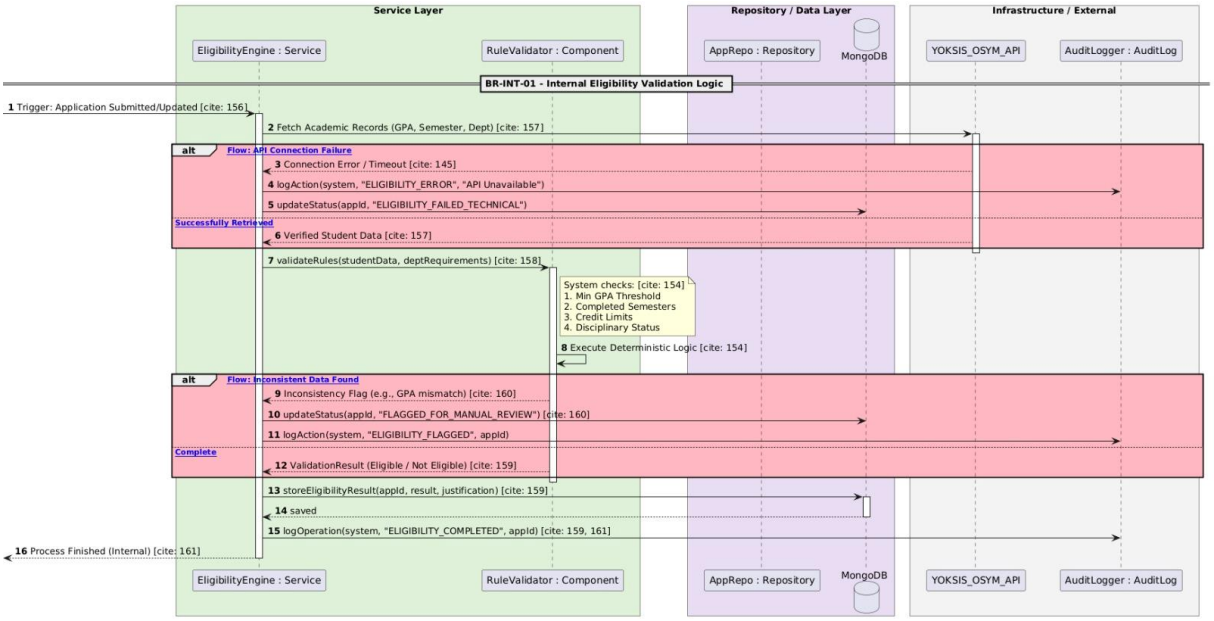
4.4.2.13 RK.1 – Approve or Adjust Ranking



4.4.2.14 CE.1 – Course-Equivalence (İntibak)



4.4.2.15 BR-INT.01 – Automatic Eligibility Check



5 Summary

The document describes the design of the **Undergraduate Transfer Management System (UTMS)**, a centralized web-based platform developed for IZTECH to digitize and automate the undergraduate transfer process.

Purpose and Scope

The SDD provides a structured overview of the system's architecture, components, and workflows. It serves as a technical blueprint for developers while ensuring compliance with **YÖK regulations** and **KVKK (data protection law)**. The system supports students, administrative staff (ÖİDB), and academic commissions (YGK) throughout the transfer process.

System Overview

UTMS manages the full lifecycle of transfer applications, including:

- Application submission
- Document upload and verification
- Automated eligibility checks (e.g., CGPA, quotas)
- Academic evaluation and ranking
- Final decision and result publication

Architecture

The system follows a **layered and modular architecture**:

- **Controller Layer:** Handles user requests
- **Service Layer:** Implements business logic
- **Model Layer:** Manages data structures
- **Database Layer:** Stores structured data (PostgreSQL)

It uses **MVC design pattern** and integrates with external systems like **UBYS API**.

Key Technologies

- **Frontend:** React + Vite
- **Backend:** Node.js
- **State Management:** Zustand
- **Database:** PostgreSQL
- **ORM:** Drizzle ORM
- **Authentication:** JWT

These technologies were selected for scalability, performance, and maintainability.

Data and Information Management

The system uses a **relational database model** to ensure:

- Data integrity and consistency
- Secure handling of sensitive data
- Traceability through audit logs
- Proper relationships between users, applications, and documents

Stakeholders

Key stakeholders include:

- Students (applicants)
- ÖİDB staff (document verification)
- YGK members (academic evaluation)
- Faculty boards (final decisions)
- Development team (project manager, developers, analysts)

Interaction Flow

The system defines detailed workflows such as:

- Login & registration
- Application submission
- Document upload & verification
- Academic evaluation
- Ranking and final approval
- Result publication

Design Goals

- **Security:** Role-based access control and KVKK compliance
- **Scalability:** Handles high application volumes
- **Maintainability:** Modular and extensible design
- **Efficiency:** Optimized for complex evaluation processes